

derp-rs

高性能 Rust DERP 中继服务器

项目设计、协议兼容与性能评估报告

项目名称	derp-rs高性能 Rust DERP 中继服务器
项目成员	郑则粲
学号	24344151
项目类型	网络系统软件 / 开源协议兼容实现
主要语言	Rust 2024 Edition
对比基线	Tailscale derper v1.100.0
报告日期	2026年 7 月
项目仓库	github.com/jaisonZheng/derp-rs

摘要

DERP (Designated Encrypted Relay for Packets) 是 Tailscale 在节点无法建立直接 WireGuard 通道时使用的中继协议。本项目以公开协议、官方源码和测试为兼容基线，使用 Rust 从零实现生产可用的高性能 DERP 服务器。系统覆盖 DERP v2 加密握手、全部稳定帧类型、Fast Start、标准 HTTP Upgrade、WebSocket、STUN、区域 Mesh、重复连接、反向路径通知、准入控制、限速、背压、TLS、可观测性和优雅重启。

实现采用 Tokio 异步运行时、DashMap 活跃连接索引、有界 MPSC 队列、bytes::Bytes 引用计数载荷和约 64 KiB 的批量写缓冲。兼容性方面，将 Tailscale 官方 Go 测试中可观察的线缆行为改写为外部服务器黑盒测试，并使用官方 Go 客户端 v1.100.0 验证；本机、GitHub Actions 和腾讯云 Linux 环境均为 12/12 通过。

性能实验显示，在 Apple M3 loopback、16 个官方客户端、1,200 字节载荷的五轮测试中，derp-rs 中位吞吐为 366,442 packets/s，官方 Go derper 为 263,363 packets/s，前者高 39.14%。在腾讯云 2 vCPU Linux 主机上，针对 100 到 10,000 条连接以及明文、TLS、持续转发、慢接收端和连接抖动等工作负载，derp-rs 在所有已测场景中的稳态与峰值 RSS 均显著低于官方实现。

关键词：DERP；Rust；Tailscale；异步网络；协议兼容；高并发；内存优化

目录

摘要	I
1 项目背景与目标	1
1.1 项目背景	1
1.2 项目目标	1
1.3 范围与边界	1
2 官方功能覆盖	1
2.1 核心能力矩阵	1
2.2 帧类型覆盖	3
3 系统设计与实现思路	3
3.1 总体架构	3
3.2 连接与加密握手	3
3.3 高性能转发路径	4
3.4 连接生命周期与内存优化	4
3.5 Mesh 与故障传播	4
4 测试设计	5
4.1 多层验证策略	5
4.2 黑盒一致性结果	5
5 实验结果	6
5.1 吞吐实验方法	6
5.2 吞吐结果	6
5.3 生产规模 RSS 实验方法	6
5.4 生产规模 RSS 结果	7
5.5 优化前后效果	7
5.6 实验结论边界	7
6 开源项目参考说明	8
6.1 Tailscale	8
6.2 Rust 社区实现	8
6.3 主要 Rust 依赖	9
7 总结与展望	9
A 复现实验命令	10
B 参考资料	10

1 项目背景与目标

1.1 项目背景

WireGuard节点通常优先通过 UDP 建立端到端直连，但对称 NAT、严格防火墙、运营商网络和受限企业网络可能使打洞失败。DERP 在这种情况下提供可靠的 TCP/TLS 中继路径。中继服务器只依据 NodeKey 转发已经端到端加密的数据包，不能解密 WireGuard 业务载荷，因此它既是可用性基础设施，也是高连接密度的网络服务。

官方 derper 由 Go 实现，功能成熟、生态完整。本项目选择 Rust 的原因不是简单重写，而是验证以下工程命题：在保持协议兼容和生产功能的前提下，是否可以利用 Rust 的所有权模型、紧凑数据结构、无垃圾回收运行时和显式背压，获得更高吞吐和更稳定的内存占用。

1.2 项目目标

1. 协议完整性：实现 DERP v2 加密握手与全部稳定帧 0x01 到 0x15，兼容官方 Tailscale 客户端。
2. 生产功能：支持 TLS、WebSocket、STUN、区域 Mesh、准入控制、速率限制、有界队列、Prometheus 指标与优雅重启。
3. 高性能：降低热路径分配、复制、锁竞争和系统调用次数，提高 relay throughput。
4. 低内存：在 100 到 10,000 条连接、慢客户端和 churn 场景下，保持可预测的稳态与峰值 RSS。
5. 可验证：使用官方 Go 客户端和公开测试行为进行黑盒互操作测试，保留可重复运行的基准脚本与机器可读原始数据。

1.3 范围与边界

本项目覆盖 DERP 线协议和服务核心行为。官方 cmd/derper 中的 ACME/GCP 证书管理、本机 tailscaled 调用、Linux XDP 快速路径和实验性 ACE proxy 属于平台集成或实验功能，不是稳定 DERP v2 互操作的必要条件。derp-rs 使用 PEM 证书、通用 HTTP Admission Controller 和静态 bootstrap DNS 文件替代相应平台绑定。

2 官方功能覆盖

2.1 核心能力矩阵

模块	已实现的官方行为	状态
连接入口	原生 Upgrade: DERP、Fast Start 与 WebSocket derp 子协议	完整
加密身份	ServerKey、NodeKey、NaCl crypto_box、ClientInfo 与 ServerInfo	完整
DERP v2 转发	NodeKey 寻址、接收包源 NodeKey、64 KiB 最大载荷、未知帧安全跳过	完整
反向路径	源节点最后一条连接断开时向既有接收者发送 PeerGone(Disconnected)	完整
目的端缺失	普通包静默，合法 disco wrapper 返回 PeerGone(NotHere)，每连接 3 次/秒	完整
重复连接	同 NodeKey 多会话、Health 状态、最新活动会话选择与恢复通知	完整
控制帧	Ping、Pong、KeepAlive、NotePreferred、Health 与 Restarting	完整
区域 Mesh	PSK、WatchConns、PeerPresent/Gone、ForwardPacket 与 ClosePeer	完整
STUN	Tailscale Binding Request 校验与 IPv4/IPv6 XOR-MAPPED-ADDRESS	完整
HTTP 运维	probe、latency-check、generate_204、bootstrap-dns、metrics 与 debug	完整
安全和资源	PEM TLS、Admission Controller、令牌桶、有界队列和写超时	完整

2.2 帧类型覆盖

值	类型	值	类型	值	类型
0x01	ServerKey	0x06	KeepAlive	0x11	ClosePeer
0x02	ClientInfo	0x07	NotePreferred	0x12	Ping
0x03	ServerInfo	0x08	PeerGone	0x13	Pong
0x04	SendPacket	0x09	PeerPresent	0x14	Health
0x05	RecvPacket	0x0a	ForwardPacket	0x15	Restarting
		0x10	WatchConns		

表 2: DERP 稳定帧覆盖

3 系统设计与实现思路

3.1 总体架构

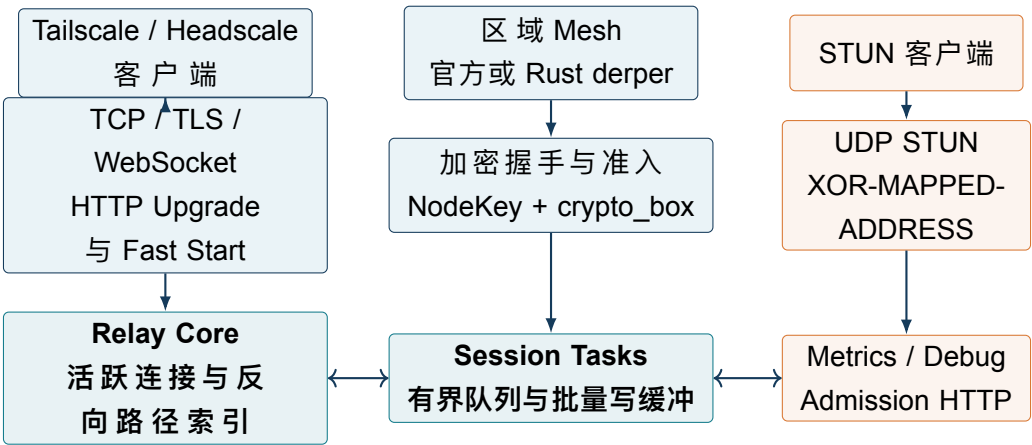


图 1: derp-rs总体架构

TCP 主监听器接受连接后，根据配置可先执行 rustls TLS 握手。HTTP 层识别 Fast Start、标准 DERP Upgrade 或 WebSocket，并将统一的异步字节流交给 DERP 会话层。UDP STUN 使用独立任务，不占用 DERP TCP 会话资源。Metrics、debug 和 admission 复用轻量 HTTP 处理逻辑。

3.2 连接与加密握手

1. 服务器发送包含固定 magic 和 32 字节公钥的 ServerKey 帧。
2. 客户端生成或加载 NodeKey，将 ClientInfo 使用双方 Curve25519 密钥和 24 字节 nonce 加密后发送。
3. 服务器解密并校验协议版本、Mesh PSK 和客户端能力；如配置 admission，则把

NodePublic 与来源地址提交给控制器。

4. 服务器注册会话，返回加密 ServerInfo，之后进入稳态 frame 循环。

NodeKey 是路由地址而不是业务内容密钥。DERP 只处理外层已加密数据包，无法读取内部 WireGuard 流量。Mesh PSK 使用常量时间比较，持久服务器私钥首次创建时采用原子写入和 0600 权限。

3.3 高性能转发热路径

- 双层索引：DashMap 保存 NodeKey 到当前活跃 Session 的热路径映射；同 key 多会话的控制面使用小型锁保护，避免每个数据包都遍历连接集合。
- 少复制载荷：数据包进入服务端后使用 `bytes::Bytes`，本地转发与 Mesh 尝试共享引用计数内存。
- 显式背压：每个客户端使用固定深度的 MPSC 队列；满队列立即计数并丢弃，慢客户端不会让服务器内存无限增长。
- 批量写：写任务将多个 frame 直接编码到同一个约 64 KiB 缓冲区，减少临时 payload、二次复制和小系统调用。
- 锁与计数：Prometheus 指标采用原子计数；本地转发查找不持有全局互斥锁。

3.4 连接生命周期与内存优化

原始 Rust 版本已经比 Go 基线节省内存，但高连接数实验显示仍可进一步优化。最终实现做了以下调整：

- 无数据的空闲连接不分配批量写缓冲；
- 多帧直接编码到单一缓冲，不再先生成临时 payload 再复制；
- 写缓冲按观测批次增长，连续 15 秒无数据后释放过大容量；
- 单批约 64 KiB，避免聚合导致不可控延迟和峰值；
- 删除两个冗余的 per-session 原子堆分配；
- 常见的单会话 PeerSet 使用 inline small-vector 存储。

3.5 Mesh 与故障传播

受信任的区域节点通过加密 ClientInfo 携带 Mesh PSK，之后使用 WatchConns 订阅 PeerPresent 与 PeerGone。目标节点不在本机时，Relay Core 查询 Mesh 路由并发送 ForwardPacket。已经转发过的包不会再次进入 Mesh，避免环路。成功的 A 到 B 转发记录反向路径；A 最后一条连接离开后，B 收到 PeerGone(Disconnected)。

4 测试设计

4.1 多层验证策略

层级	主要内容	作用
Rust 单元测试	frame round-trip、官方 greeting 向量、crypto box、路由、重复连接、限速、STUN	快速验证纯逻辑和边界条件
Rust 集成测试	WebSocket 握手与 Ping/Pong、Rust 到 Rust regional mesh	验证异步服务组合
官方客户端黑盒	官方 Go DERP HTTP 客户端连接真实 release 进程	避免“服务端和自制客户端同时写错”
社区行为适配	双向转发、最新连接路由、非 Fast Start Upgrade	补充独立 Rust 社区实现视角
生产压力测试	100 到 10,000 连接、TLS、active、slow、churn	验证资源边界和内存峰值

表 2: 验证层级

Tailscale 上游测试通常把 Go derpserver.Server 直接嵌入测试进程，不能直接指向第三方二进制。本项目将可观察的协议行为改写为外部服务器测试，官方 Go DERP HTTP 客户端固定为 v1.100.0。Go 内部数据结构、XDP 实现细节和只测试客户端错误处理的用例不做机械移植。

4.2 黑盒一致性结果

环境	结果
Apple Silicon macOS，本机 release 进程	12/12 通过
腾讯云 Ubuntu 24.04 x86_64，CI release ELF	12/12 通过
GitHub Actions Ubuntu，Rust 测试 + Clippy + 黑盒套件	全部通过

表 3: 跨平台一致性验证

黑盒套件覆盖三节点转发、双向转发、源 NodeKey、Ping/Pong、反向 PeerGone、NotHere disco 规则和 burst 限速、Mesh watcher 快照与上下线、重复连接 Health、最新连接路由、NotePreferred、Fast Start、标准 HTTP 101 Upgrade、运维端点和官方 STUN 请求/解析。

5 实验结果

5.1 吞吐实验方法

- 硬件：Apple M3，macOS 15.6，arm64。
- 服务端：derp-rs 0.1.0 release 与官方 derper v1.100.0。
- 客户端：16 个官方 Go DERP 客户端组成转发环。
- 负载：1,200 字节 payload，每轮 256,000 个确认送达的数据包。
- 传输：loopback TCP、Fast Start、关闭 TLS 与 STUN。
- 统计：每个实现运行 5 次，报告 packets/s 中位数。

负载生成器只有在接收端验证所有发送包后才推进窗口，因此结果不能通过静默丢包获得。该实验强调 framing、路由、分配、队列、调度和 socket write，不代表公网 RTT。

5.2 吞吐结果

服务端	五轮 packets/s	中位数	有效载荷
derp-rs 0.1.0	380,213; 366,442; 375,886; 289,351; 312,408	366,442	3.518 Gbit/s
Go derper v1.100.0	259,203; 280,457; 256,018; 263,363; 286,846	263,363	2.528 Gbit/s

表 4: 同机五轮 DERP 转发吞吐

derp-rs 的中位吞吐比官方实现高 **39.14%**。同一微基准中的 RSS 样本为 4,704 KiB 对 22,208 KiB，Rust 低 78.82%，Go 进程为 Rust 的 4.72 倍。RSS 样本不是峰值，因此更强的内存结论来自下一节独立的 Linux 生产规模矩阵。

5.3 生产规模 RSS 实验方法

实验运行于腾讯云 Ubuntu、Linux 6.8 x86_64、2 vCPU、1.92 GiB RAM 主机。服务器每个场景重新启动，客户端使用官方 [tailscale.com/derp v1.100.0](https://tailscale.com/derp)：

- **idle**：完成握手并持续读取控制帧；
- **active**：客户端组成环，以总计约 10,000 packets/s 转发并验证接收；
- **slow**：一半接收端停止读取且 socket receive buffer 为 4 KiB；
- **churn**：每 500 ms 关闭并替换约 10% 活跃连接。

预热后，baseline 为 10 次 100 ms 采样均值，steady 为状态就绪后的 20 次 100 ms 采样均值，peak 每 50 ms 采样一次。Linux 数据源为 `/proc/<pid>/status` 的 VmRSS。

5.4 生产规模 RSS 结果

传输	工作负载	连接	Rust/MiB	Go/MiB	降幅
明文	Idle	1,000	15.9	51.5	69.17%
明文	Idle	10,000	111.4	358.9	68.96%
明文	Active	5,000	70.0	204.1	65.72%
明文	Slow receiver	5,000	111.9	308.1	63.69%
明文	Churn	5,000	58.3	202.9	71.26%
TLS	Idle	1,000	24.0	79.1	69.67%
TLS	Idle	10,000	184.9	595.1	68.93%
TLS	Active	5,000	107.3	324.7	66.95%
TLS	Slow receiver	5,000	156.8	439.2	64.30%
TLS	Churn	5,000	96.0	314.4	69.47%

表 5: 选定场景的稳态 RSS

完整明文矩阵中，derp-rs 稳态 RSS 低 **62.38%** 到 **71.36%**，峰值低 **62.38%** 到 **79.85%**；完整 TLS 矩阵中，稳态低 **63.65%** 到 **70.99%**，峰值低 **63.65%** 到 **78.91%**。

5,000 连接 churn 能体现垃圾回收和连接生命周期造成的峰值差异。明文 peak RSS 为 Rust 58.6 MiB、Go 290.8 MiB；TLS 为 Rust 100.1 MiB、Go 391.4 MiB。慢接收端场景中有界队列有效限制了无法及时写出的 payload 数量。

5.5 优化前后效果

同一批优化还将本地五轮吞吐中位数从 313,906 提升到 366,442 packets/s，说明内存下降不是通过降低转发工作量换取的。

5.6 实验结论边界

实验支持以下结论：在指定 Linux 主机和 100 到 10,000 条连接的测试包络内，包括 TLS、持续流量、背压和 churn，derp-rs 的稳态与峰值 RSS 始终显著低于官方 derper v1.100.0。该结论不是对所有操作系统、分配器、CPU 架构、未来 Go/Rust 版本、扩展配置、任意流量分布或无限连接数的数学保证。公网性能还会受到带宽、丢包、拥塞和 RTT 主导。生产容量规划应在目标主机重跑仓库脚本。

传输	5,000连接	原始 Rust/MiB	优化 Rust/MiB	降幅
明文	Idle	78.0	58.6	24.84%
明文	Active	90.2	70.0	22.40%
明文	Slow receiver	134.4	111.9	16.74%
TLS	Idle	115.6	95.6	17.30%
TLS	Active	127.0	107.3	15.51%
TLS	Slow receiver	172.3	156.8	9.01%

表 6: 内存优化前后稳态 RSS

6 开源项目参考说明

6.1 Tailscale

本项目最主要的兼容参考是 Tailscale 公开仓库中的 DERP 协议、Go 客户端、derpserver、cmd/derper 与 STUN 实现。开发时检查的主线提交如下，稳定互操作和性能基线固定为 v1.100.0。

cfd101f9d773695def27a5f6289fc25ac36ac991

测试参考包括：

- derp/derp_test.go: SendRecv、Watch、Ping/Pong、PeerGone；
- derp/derphttp/derphttp_test.go: 官方 HTTP 客户端与 probe；
- derp/derpserver/derpserver_test.go: 重复连接、Mesh、限速；
- net/stun/stun_test.go: STUN request/response 向量。

derp-rs 是依据公开协议和可观察行为编写的独立实现，不包含 Tailscale 商标授权，也不自称 Tailscale 官方产品。项目采用与上游兼容的 BSD-3-Clause 许可证。

6.2 Rust 社区实现

rajsinghtech/rustscale 的测试型 DERP server 提供了双向转发、latest-writer 路由和非 Fast Start Upgrade 测试思路。本项目移植其与官方协议一致的行为。该社区实现要求新连接关闭同 key 旧连接，而当前 Tailscale 官方行为是保留重复连接并发送 Health，因此冲突部分以官方语义为准，没有机械照搬。

开源组件	在项目中的用途
Tokio	多线程异步运行时、TCP/UDP、任务、channel、signal 和 timer
rustls / tokio-rustls	无 OpenSSL 运行时依赖的 TLS 服务器
crypto_box	Curve25519 XSalsa20Poly1305 兼容加密盒
DashMap / parking_lot	并发连接索引与轻量控制面锁
bytes	引用计数的网络载荷及缓冲管理
tokio-tungstenite	WebSocket DERP 子协议
serde / serde_json	ClientInfo、ServerInfo、admission 和配置数据
clap / tracing	命令行参数与结构化运行日志

表 7: 主要开源依赖及用途

6.3 主要 Rust 依赖

7 总结与展望

本项目完成了一个协议兼容、功能覆盖完整、可独立部署的 Rust DERP 服务器。与官方 Go derper 的对比表明，显式内存生命周期、有界背压和批量写设计能够在不牺牲协议正确性的情况下，同时提升吞吐并显著降低连接密度下的 RSS。

后续工作包括：在更多 CPU 架构和云厂商复现实验；增加长时间故障注入、网络丢包和模糊测试；对 admission 与 Mesh 管理面开展独立安全审计；根据上游协议演进持续运行黑盒兼容套件。对生产部署而言，仍应结合目标网络的 RTT、带宽和连接分布进行容量规划。

A 复现实验命令

```
cargo test --all-targets --locked
cargo clippy --all-targets --locked -- -D warnings
./scripts/official-conformance.sh
./scripts/benchmark.sh
./scripts/rss-benchmark.sh
```

完整生产矩阵可使用：

```
TLS_MODE=1 \
SCENARIOS="idle:100 idle:1000 idle:5000 idle:10000 \
active:1000 active:5000 slow:1000 slow:5000 \
churn:1000 churn:5000" \
./scripts/rss-benchmark.sh
```

B 参考资料

参考文献

- [1] Tailscale Inc., *DERP protocol package*,
<https://pkg.go.dev/tailscale.com/derp>.
- [2] Tailscale Inc., *DERP server implementation*, <https://github.com/tailscale/tailscale/tree/main/derp/derpserver>.
- [3] Tailscale Inc., *cmd/derper*,
<https://github.com/tailscale/tailscale/tree/main/cmd/derper>.
- [4] Tailscale Inc., *STUN implementation*,
<https://github.com/tailscale/tailscale/tree/main/net/stun>.
- [5] Raj Singh, *rustscale DERP test server*, <https://github.com/rajsinghtech/rustscale/blob/master/crates/derp/src/server.rs>.
- [6] Jason A. Donenfeld, *WireGuard: Next Generation Kernel Network Tunnel*,
<https://www.wireguard.com/papers/wireguard.pdf>.